

# Polymarket Gamma Starter

## v3.3.0-real Operators Manual and In-Depth Guide

Complete Operator UX Redesign, Performance Polish, and Interaction Overhaul

Generated from package: polymarket-gamma-starter-v3.3.0-real.zip

Package SHA-256: 302630a3cba85dbb39c47e345a73c40350f17ae7d46e010ddfeed71e2e296fa5

### Primary safety statement

This guide documents a local-first, human-in-the-loop operator workstation. It does not provide financial advice. Application outputs, analytics, reports, search results, graph findings, workflows, packets, and demo data are not orders and are not execution approvals. Live order submission remains guarded by backend gates, risk controls, warning acknowledgement, typed confirmation, read-only state, kill switch state, Live Armed mode, and explicit operator action.

Prepared: 2026-06-17 03:02 UTC

# Table of Contents

Table of Contents

2

1. Purpose, Scope, and Operating Model

6

What the software is . . . . .

6

What the software is not . . . . .

6

2. Safety Model and Non-Negotiable Boundaries

6

Live submission gate chain . . . . .

7

Cancel gate chain . . . . .

7

Secret handling . . . . .

7

3. Installation and First Launch

7

Recommended local setup . . . . .

7

First login and admin setup . . . . .

8

Important paths . . . . .

8

4. Configuration and Environment Management

8

Most important safe defaults . . . . .

8

Configuration workflow . . . . .

9

5. System Architecture Overview

9

Major subsystems . . . . .

9

Local-first data philosophy . . . . .

9

6. The v3 Operator Interface

10

Navigation groups . . . . .

10

Using the Command Center . . . . .

10

7. Global Local Search and Decision Graph

10

Global local search . . . . .

10

Decision/object graph . . . . .

11

8. Read-Only Workflows, Packets, and Briefs

11

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| Core v3 workflows . . . . .                                            | 11        |
| How to read a packet . . . . .                                         | 11        |
| <b>9. Operator Analytics and Learning Reports</b>                      | <b>12</b> |
| Analytics categories . . . . .                                         | 12        |
| <b>10. Live v2 Control Plane</b>                                       | <b>12</b> |
| Primary Live v2 pages . . . . .                                        | 12        |
| Pre-submit discipline . . . . .                                        | 13        |
| <b>11. Strategy, Theses, Evidence, Watchlists, and Scorecards</b>      | <b>13</b> |
| Recommended thesis workflow . . . . .                                  | 13        |
| <b>12. Research Intake, Source Workflow, and Evidence Freshness</b>    | <b>13</b> |
| Core research objects . . . . .                                        | 14        |
| Human-controlled evidence conversion . . . . .                         | 14        |
| <b>13. Monitoring and Alert Workflow</b>                               | <b>14</b> |
| Alert rule lifecycle . . . . .                                         | 14        |
| Common alert types . . . . .                                           | 14        |
| <b>14. Portfolio and Exposure Intelligence</b>                         | <b>14</b> |
| Exposure review flow . . . . .                                         | 15        |
| <b>15. Governance, Decision Journal, Reviews, and Mistake Patterns</b> | <b>15</b> |
| Decision journal fields to use seriously . . . . .                     | 15        |
| Pre-trade governance checklist . . . . .                               | 15        |
| <b>16. Data Integrity, Backup, Restore, Import, and Migration</b>      | <b>16</b> |
| Backup discipline . . . . .                                            | 16        |
| Restore discipline . . . . .                                           | 16        |
| <b>17. Safe Demo Mode, Visual QA, and Release Validation</b>           | <b>16</b> |
| Demo commands . . . . .                                                | 16        |
| Release validation commands . . . . .                                  | 16        |
| <b>18. Reports, Exports, and Documentation Outputs</b>                 | <b>17</b> |

|                                                          |           |
|----------------------------------------------------------|-----------|
| Common export categories . . . . .                       | 17        |
| Export rules . . . . .                                   | 17        |
| <b>19. Recommended Operator Playbooks</b>                | <b>17</b> |
| Daily startup playbook . . . . .                         | 17        |
| Research-to-thesis playbook . . . . .                    | 17        |
| Pre-trade review playbook . . . . .                      | 18        |
| Post-trade or post-decision review playbook . . . . .    | 18        |
| Weekly learning playbook . . . . .                       | 18        |
| <b>20. Troubleshooting and Recovery</b>                  | <b>18</b> |
| App will not start . . . . .                             | 18        |
| Cannot log in . . . . .                                  | 19        |
| Data looks missing . . . . .                             | 19        |
| Live readiness fails . . . . .                           | 19        |
| <b>21. Route and API Reference</b>                       | <b>19</b> |
| v3 UI routes . . . . .                                   | 19        |
| v3 API route families . . . . .                          | 20        |
| v2-live module route families . . . . .                  | 20        |
| <b>22. Appendix: Environment Variable Reference</b>      | <b>20</b> |
| Server and security . . . . .                            | 20        |
| Live and read-only controls . . . . .                    | 21        |
| Risk and paper settings . . . . .                        | 21        |
| Data ingestion and training . . . . .                    | 21        |
| Credential placeholders . . . . .                        | 22        |
| <b>23. Appendix: Maintenance, Upgrades, and Rollback</b> | <b>22</b> |
| Before upgrading . . . . .                               | 22        |
| After upgrading . . . . .                                | 22        |
| Rollback . . . . .                                       | 22        |

**24. Closing Operator Notes**

**22**

# 1. Purpose, Scope, and Operating Model

This manual is a practical operator guide for Polymarket Gamma Starter v3.3.0-real. It is written for someone who needs to install the application, understand its safety model, operate the v3 command center, use the detailed v2 live-control modules, manage local data, run analytics, create reports, and maintain the package without losing human control.

The application is best understood as a local-first operator workstation. It connects research, evidence, strategy, monitoring, portfolio exposure, governance, backups, visual QA, analytics, and carefully gated live-control boundaries. The goal is not to automate trading. The goal is to make the operator better informed, better organized, and less likely to act without a traceable process.

v3.3.0-real is specifically a usability and interaction-overhaul release. It preserves the underlying modules and safety gates while improving the app shell, navigation, design system, command center, reports, forms, tables, and visual QA workflow.

## Use this manual as an operating reference

Read sections 2 through 6 before attempting any live-connected configuration. Use the playbooks in section 19 for day-to-day operation. Use the appendices when configuring environment variables, validating a release, or troubleshooting.

## What the software is

- A local-first Polymarket research, operations, and safety workstation.
- A human-in-the-loop control plane with explicit live gates.
- A structured place to manage theses, sources, evidence, monitoring rules, exposure, governance, analytics, and data integrity.
- A v3 command center that ties together the older v2 detailed workflows and newer intelligence layers.
- A reporting and review system for pre-trade packets, market briefs, thesis health, portfolio risk, operator reviews, and learning reports.

## What the software is not

- It is not a guarantee of profit or predictive accuracy.
- It is not financial advice.
- It is not an autonomous trading bot by default.
- It is not a way to bypass Polymarket restrictions, terms, jurisdictional rules, geofencing, account eligibility, KYC, or compliance obligations.
- It is not safe to use with real credentials unless the operator understands and intentionally configures every relevant gate.

# 2. Safety Model and Non-Negotiable Boundaries

The application has many read-only and local-first capabilities, but live execution is intentionally hard to reach. The design expects the operator to review data, evaluate risks, explicitly approve, acknowledge warnings, type a configured confirmation phrase, and only then pass backend gates. UI convenience does not replace backend enforcement.

**Do not confuse intelligence with authorization**

Search results, graph findings, analytics scores, learning reports, pre-trade packets, market briefs, thesis health reports, portfolio briefs, monitoring alerts, governance checklists, and demo data never authorize order placement. They are review aids only.

## Live submission gate chain

The v2 live-trading documentation describes the fail-closed submit path. A live order is blocked unless every configured live gate passes. The exact implementation can evolve, but the operator should treat the following as mandatory concepts:

- Trading mode must be intentionally set to an armed live mode.
- Read-only mode must not be forcing execution disabled.
- The kill switch must be off.
- Real network permission must be enabled intentionally.
- Submit or cancel capability must be explicitly enabled.
- Required credentials and optional live SDK dependencies must be present locally.
- Risk checks must pass.
- The operator must explicitly provide human approval.
- Warnings must be acknowledged.
- The typed confirmation phrase must exactly match the configured phrase.

## Cancel gate chain

Cancellation is also gated. It requires explicit cancel permission, real-network permission, a valid order identifier, a reason, and the configured typed confirmation phrase. Do not assume a UI cancel button is enough. Backend checks remain authoritative.

## Secret handling

- Do not commit .env files with real secrets.
- Do not paste wallet private keys or API secrets into reports, screenshots, tickets, or issue trackers.
- Use .env.example only as a template with placeholders.
- Backups and exports are designed to redact or exclude secrets by default.
- AI/model assistance is disabled by default and must never receive keys, wallet secrets, auth headers, or private account data.

# 3. Installation and First Launch

The package is a Python/FastAPI-style web application with server-rendered templates and local runtime state under data directories. The normal launch path creates a virtual environment, installs requirements, and starts the local server with run.py.

## Recommended local setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python run.py
```

On Windows PowerShell, activate the environment with ``.venv\Scripts\Activate.ps1`` before installing dependencies and running the app.

## First login and admin setup

When no users exist, the app redirects to `/setup`` and requires creation of the initial admin account. The default username created by setup is `admin``; the operator chooses the password. Passwords are stored as PBKDF2 hashes in local runtime state, not as plaintext.

- Open the app in a browser after launching it.
- Create the initial admin password at `/setup``.
- Log in at `/login``.
- Use `/users`` as an admin to create read-only or admin users as needed.
- Keep the admin account secured; at least one active admin must remain.

## Important paths

| Path                                                | Purpose                                                                                            |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>/v3`</code>                                   | Primary redesigned command center. Start here for v3 operations.                                   |
| <code>/v2-live`</code>                              | Detailed Live v2 control plane and module navigation.                                              |
| <code>/settings/configuration`</code>               | Schema-backed configuration console. Prefer this over raw <code>.env</code> editing when possible. |
| <code>/setup/status`</code>                         | Runtime status, Python/venv/dependency/.env guidance.                                              |
| <code>/api/status`</code> and <code>/health`</code> | Basic service status checks.                                                                       |
| <code>/data/`</code>                                | Local runtime state. Excluded from release ZIPs. Back up carefully.                                |

# 4. Configuration and Environment Management

The `.env.example`` file is a safe template, not a place for real credentials. The GUI configuration console and setup wizard are the preferred operator path for common settings. Raw environment edits are still supported but should be done carefully with backups and secret hygiene.

## Most important safe defaults

| Setting                                  | Default intent                           | Operator meaning                                                         |
|------------------------------------------|------------------------------------------|--------------------------------------------------------------------------|
| APP_MODE=read_only                       | Safe local review mode                   | The app starts in read-only posture unless explicitly changed.           |
| READ_ONLY=true                           | Blocks live mutation                     | Keep true for research, demos, analytics, QA, and most normal operation. |
| LIVE_TRADING_ENABLED=false               | Live disabled by default                 | Do not enable casually. This is only one of several required gates.      |
| POLYMARKET_V2_TRADING_MODE=research_only | v2 live control plane is safe by default | Switch only after understanding all gates and compliance obligations.    |
| POLYMARKET_LIVE_KILL_SWITCH=true         | Emergency stop defaults to active        | Must be intentionally disabled before live mutation could pass.          |



| Setting                                      | Default intent                          | Operator meaning                                               |
|----------------------------------------------|-----------------------------------------|----------------------------------------------------------------|
| POLYMARKET_LIVE_ALLOW_REAL_NETW<br>ORK=false | No real network mutation by<br>default  | Validation and normal use do not submit/cancel orders.         |
| POLYMARKET_DATA_ALLOW_NETWORK=f<br>alse      | No automatic data network by<br>default | Internet/data ingestion must be explicitly enabled and scoped. |

## Configuration workflow

- Start from ``.env.example`` and create a private local ``.env`` outside source control.
- Use `~/setup/status`` to inspect runtime and dependency status.
- Use `~/setup/wizard`` to apply safe presets for local demo, LAN demo, training/backtesting, or high-volume host training.
- Use `~/settings/configuration`` for schema-backed edits, validation, diff previews, sanitized export, and audit history.
- Before changing live-related settings, export a sanitized config and create a runtime backup.
- After any material change, run the release or startup smoke validation scripts.

### LAN access warning

HOST=0.0.0.0 makes the server reachable from other machines on the LAN. Use this only on trusted networks, with authentication enabled, and without exposing secrets. Use HOST=127.0.0.1 for local-only operation.

## 5. System Architecture Overview

At a high level, the application is a Python web app with HTML templates, static CSS/JavaScript, local JSON/JSONL runtime files, and explicit adapter boundaries for read-only network and live execution. The v3 layer is an operator shell and intelligence layer that reads from local subsystems. The v2 layer remains the detailed control plane for live-related workflows.

### Major subsystems

| Subsystem      | Core module(s)                                 | Primary responsibility                                                                      |
|----------------|------------------------------------------------|---------------------------------------------------------------------------------------------|
| Live control   | live_v2.py, live_trading.py,<br>live_* modules | Readiness, tickets, risk checks, order preview, gated submit/cancel, reconciliation, audit. |
| Strategy       | live_strategy.py                               | Theses, evidence linkage, watchlists, scorecards, review, ticket draft context.             |
| Research       | live_research.py                               | Source registry, research queue, notes, evidence candidates, freshness, thesis comparison.  |
| Monitoring     | live_monitoring.py                             | Alert rules, manual evaluation, active alerts, acknowledgement, snooze, exports.            |
| Portfolio      | live_portfolio.py                              | Exposure summaries, bankroll/risk budgets, scenarios, planned impact.                       |
| Governance     | live_governance.py                             | Decision journal, checklists, reviews, rules, near-misses, mistake patterns.                |
| Data integrity | live_data.py                                   | Runtime inventory, health checks, secret scan, backups, restore preview, migrations.        |
| v3 OS          | live_v3.py                                     | Command center, search, graph, workflows, packets, briefs, settings, demo support.          |
| v3 Analytics   | live_v3_analytics.py                           | Decision/thesis/evidence/alert/governance/portfolio analytics and learning reports.         |

### Local-first data philosophy

Most operator records are local runtime state. This makes the workstation auditable and portable, but it also makes backup discipline important. Release ZIPs should not contain runtime records, real audit ledgers, screenshots, backups,

or credentials. Data health, backup, restore, import, and migration tools are provided to make local state manageable.

## 6. The v3 Operator Interface

v3.3.0-real redesigns the interface around an operator shell with grouped navigation, consistent cards and status badges, clearer safety hierarchy, improved command center, search and graph explorers, workflow cards, and polished report layouts. The v3 shell is intended to be the operator's default home.

### Navigation groups

| Group        | Use it for                                                           | Important pages                                                                |
|--------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Operate      | Immediate operational status and live-control workflow entry points. | /v3, /v2-live, trade ticket, audit.                                            |
| Analyze      | Analytics, portfolio risk, monitoring, graph, and search.            | /v3/analytics, /v3/graph, /v3/search, /v2-live/portfolio, /v2-live/monitoring. |
| Build Thesis | Strategy and research development.                                   | /v2-live/strategy, /v2-live/research, evidence, watchlists.                    |
| Govern       | Decision journal, reviews, backups, settings, data safety.           | /v2-live/governance, /v2-live/data, /v3/settings.                              |
| Output       | Briefs, reports, workflows, docs, exports.                           | /v3/workflows, /v3/briefs, /v3/docs.                                           |

### Using the Command Center

- Open `/v3` first after login.
- Read System Safety before doing anything else.
- Review the Operator Attention Queue for blockers, stale evidence, unresolved research, alerts, concentration warnings, incomplete governance, and data health issues.
- Use Workbench Shortcuts to jump into the next safe workflow.
- Use Intelligence Summary for high-level analytics context, not trade signals.
- Use Recent Activity to understand what changed locally.
- Treat Safe Next Actions as review prompts, not financial advice or execution instructions.

#### Command Center rule

The command center is read-only by design. It summarizes state and suggests operator review actions. It does not submit, cancel, approve, or arm orders.

## 7. Global Local Search and Decision Graph

The global search and graph make local operator context discoverable. They connect records across modules so the operator can find a thesis, research source, alert, portfolio warning, governance review, or analytics report without remembering exactly where it was created.

### Global local search

- Search is local-first and does not send local data to external services.
- Use filters for object type, status, tag, and recentness where available.
- Use search for retrieval and navigation, not recommendation.

- Rebuild the index when demo data or large local records change.
- Export only sanitized search summaries when needed.

## Decision/object graph

The graph is implemented as a stable relationship explorer rather than a fragile visual-only network. It links nodes such as markets, theses, evidence, sources, alerts, exposure, scenarios, reviews, trade tickets, orders, audit events, data health checks, analytics snapshots, and learning reports.

| Relationship type    | Meaning for operators                                                  |
|----------------------|------------------------------------------------------------------------|
| supports             | Evidence or record supports a thesis, decision, or review.             |
| contradicts          | Evidence or record contradicts a thesis or decision.                   |
| blocked_by           | Object has a blocker requiring review before proceeding.               |
| alerted_by           | Monitoring alert relates to the object.                                |
| governed_by          | Governance checklist, rule, or review relates to the object.           |
| audited_by           | Audit event records the action.                                        |
| derived_from         | Report, packet, or analytics object was generated from source records. |
| missing_prerequisite | A required or expected dependency is absent or incomplete.             |

## 8. Read-Only Workflows, Packets, and Briefs

The v3 workflow orchestrator produces structured operator outputs. These outputs are designed to reduce context-switching and improve review discipline. They must remain read-only and draft-like unless a separate, explicit workflow says otherwise. They do not place, cancel, or approve orders.

### Core v3 workflows

| Workflow                      | Primary purpose                                                                                                                                       | Typical use                                                |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Pre-Trade Intelligence Packet | Assemble market, thesis, evidence, alert, portfolio, governance, data health, and readiness context.                                                  | Run before drafting or reviewing a trade ticket.           |
| Market Intelligence Brief     | Summarize local market context, research, sources, alerts, and missing work.                                                                          | Run when deciding whether a market deserves more research. |
| Thesis Health Report          | Evaluate evidence coverage, counter-evidence, stale data, alerts, exposure, governance, and invalidation criteria.                                    | Run before marking a thesis ready.                         |
| Portfolio Risk Brief          | Summarize exposure, concentration, stale evidence exposure, and risk-process concerns.                                                                | Run before adding exposure or during weekly review.        |
| Operator Review Packet        | Summarize daily or weekly decisions, alerts, portfolio warnings, governance state, risk blocks, and next actions.                                     | Use during daily and weekly review routines.               |
| Learning Report               | Summarize decision quality, thesis quality, evidence usefulness, alert usefulness, governance discipline, calibration, mistake and strength patterns. | Use for improvement, not prediction.                       |

### How to read a packet

- Start with the status: ready, not ready, needs review, blocked, unknown, or unavailable.

- Read blockers before warnings; blockers indicate prerequisites the operator must address.
- Treat unknown/unavailable data as a reason to slow down, not as a safe assumption.
- Use source sections to understand where the packet pulled context from.
- Export packets when you need an immutable review artifact, but do not include secrets.

## 9. Operator Analytics and Learning Reports

v3.2 added the analytics layer, and v3.3 polished its presentation. Analytics are descriptive and local-first. They do not predict future market outcomes, guarantee improvement, or provide financial advice. Their purpose is to help the operator see process quality over time.

### Analytics categories

| Category               | Questions it helps answer                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Decision quality       | Are decisions reviewed? Do they link to evidence, counter-evidence, risk review, governance, exit criteria, and follow-ups? |
| Thesis quality         | Which theses are stale, incomplete, missing evidence, lacking counter-evidence, or linked to unresolved research?           |
| Evidence usefulness    | Which evidence is fresh, stale, linked, contradicted, useful, or requiring refresh?                                         |
| Alert usefulness       | Which alerts are acknowledged, useful, noisy, unresolved, or tied to decisions and near-misses?                             |
| Governance discipline  | Are checklists, reviews, near-misses, rules, and process improvements being completed?                                      |
| Portfolio/risk process | Are planned impacts evaluated? Are concentration warnings recurring? Are risk blocks respected?                             |
| Confidence calibration | How do confidence buckets compare with reviewed outcomes, and how many outcomes remain unknown?                             |

#### Sample-size warning

Calibration and pattern analytics are only meaningful when enough reviewed records exist. The app should show sample size and unknown count rather than overstate conclusions.

## 10. Live v2 Control Plane

The v2-live console remains the detailed live-control plane. v3 links to it rather than replacing it. Operators should use v3 for unified context and v2-live for detailed readiness, market discovery, ticket preview, submit/cancel gate review, positions, orders, risk, audit, settings, emergency controls, and verification.

### Primary Live v2 pages

| Page                                      | Purpose                                                      |
|-------------------------------------------|--------------------------------------------------------------|
| /v2-live                                  | Consolidated Live v2 dashboard.                              |
| /v2-live/readiness                        | Pass/fail readiness checklist.                               |
| /v2-live/markets and /v2-live/market-data | Market discovery and order book read-only links.             |
| /v2-live/trade-ticket                     | Ticket payload requirements, preview, and gated submit flow. |
| /v2-live/orders                           | Open orders and reconciliation entry points.                 |
| /v2-live/positions                        | Read-only position/balance entry points where available.     |
| /v2-live/risk                             | Risk control status and settings overview.                   |

| Page               | Purpose                                  |
|--------------------|------------------------------------------|
| /v2-live/audit     | Local audit ledger and exports.          |
| /v2-live/emergency | Emergency-control action documentation.  |
| /v2-live/verify    | Explicit read-only verification harness. |

## Pre-submit discipline

- Never use the submit endpoint as a test unless using explicit dry-run/fake paths.
- Run readiness checks first.
- Run or refresh relevant market data if needed.
- Create or link a thesis with evidence and counter-evidence review.
- Run a pre-trade intelligence packet.
- Review portfolio impact and concentration warnings.
- Complete governance checklist items.
- Only proceed if every backend gate, risk check, warning acknowledgement, and typed confirmation requirement is intentionally satisfied.

## 11. Strategy, Theses, Evidence, Watchlists, and Scorecards

The strategy layer is where trade ideas become structured theses. A thesis should capture the market context, rationale, entry criteria, exit criteria, invalidation criteria, evidence, counter-evidence, scorecard, watchlist status, and review state. It is the bridge between research and a ticket draft, but it does not submit orders.

### Recommended thesis workflow

- Create or update the thesis with a clear title, market, outcome, rationale, and status.
- Attach supporting evidence only after source review.
- Record counter-evidence or explicitly acknowledge that it is missing.
- Set entry, exit, and invalidation criteria before a ticket draft is considered.
- Create or update a scorecard if the workflow needs a structured evaluation.
- Use watchlists for candidates that need continued monitoring instead of immediate action.
- Run thesis health before moving from research to ticket review.

#### Strategy-to-ticket boundary

A strategy thesis can produce a ticket draft containing context and rationale. This is not an order. Live submission remains gated by the Live v2 backend.

## 12. Research Intake, Source Workflow, and Evidence Freshness

The research layer organizes sources, notes, research queue items, evidence candidates, freshness status, and thesis comparison. It makes it easier to distinguish what the operator has actually reviewed from what still needs verification.

## Core research objects

| Object              | Use                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------|
| Source              | A URL, document, note, dataset, or other origin that may support research.              |
| Research queue item | A work item that needs operator review or follow-up.                                    |
| Source note         | Operator note tied to a source or research question.                                    |
| Evidence candidate  | Potential evidence before conversion into strategy evidence.                            |
| Freshness record    | Review and stale-date tracking for evidence or sources.                                 |
| Thesis comparison   | Structured comparison of evidence, contradiction, and confidence context across theses. |

## Human-controlled evidence conversion

Evidence candidates should be reviewed before conversion. The conversion step can create a strategy evidence record, but it must not submit orders, alter live mode, or bypass ticket gates. This separation keeps source intake from becoming hidden execution logic.

## 13. Monitoring and Alert Workflow

Monitoring adds operator awareness. It tracks local alert rules, watchlist thresholds, thesis review reminders, evidence freshness reminders, market condition checks where read-only data is available, and readiness/safety posture alerts. Monitoring does not trade.

### Alert rule lifecycle

- Create a rule with a clear type, severity, condition, target object, and notes.
- Manually evaluate the rule or use explicitly configured safe read-only refresh if available.
- Review active alerts in the notification center.
- Acknowledge alerts when reviewed.
- Snooze alerts only when there is a concrete reason and follow-up plan.
- Archive or tune noisy rules rather than ignoring them indefinitely.
- Use alert analytics to identify useful versus noisy rules.

### Common alert types

| Type             | Examples                                                                                   |
|------------------|--------------------------------------------------------------------------------------------|
| Price threshold  | Above/below/crosses target, entry or exit levels.                                          |
| Spread/liquidity | Spread too wide, liquidity too low, order book unavailable or stale.                       |
| Market status    | Active, closed, stale, approaching resolution where detectable.                            |
| Thesis/research  | Review date reached, stale evidence, unresolved research.                                  |
| Readiness/safety | Kill switch, read-only state, live armed state, verification failure, missing credentials. |

## 14. Portfolio and Exposure Intelligence

The portfolio layer helps the operator understand exposure, concentration, bankroll usage, risk budgets, scenario impact, and planned trade impact. It distinguishes actual live exposure, read-only reported exposure, locally audited

exposure, paper exposure, planned exposure, and unknown exposure. It must not invent values when source data is unavailable.

## Exposure review flow

- Review total exposure and unknown exposure first.
- Check exposure by market, thesis, watchlist, and tag/playbook.
- Check bankroll/risk-budget settings for maximum portfolio, market, thesis, tag, daily notional, and reserve amounts.
- Review concentration warnings, especially those linked to stale evidence or unresolved research.
- Run planned trade impact before drafting or submitting tickets.
- Use scenarios to reason about what could happen if markets resolve or liquidity disappears.
- Treat unknown/unavailable data cautiously and document assumptions.

### No invented balances

The application should not infer real account balances or P&L unless safe, explicit, reliable read-only data exists. Unknown means unknown.

## 15. Governance, Decision Journal, Reviews, and Mistake Patterns

Governance makes the operator accountable to a process. It records decision journal entries, pre-trade checklists, post-trade reviews, daily and weekly reviews, governance rules, rule violations, near-misses, mistake patterns, and process improvement items.

### Decision journal fields to use seriously

- Decision title and type.
- Linked market, thesis, evidence, alert, portfolio exposure, ticket, or order when applicable.
- Decision summary and reasoning.
- Confidence level and expected outcome.
- Risk considered and alternative considered.
- Rule/checklist status.
- Follow-up date and status.
- Operator notes and tags.

### Pre-trade governance checklist

Before proceeding toward a live-related ticket, the checklist should confirm that a thesis exists, evidence and counter-evidence have been reviewed, stale evidence and unresolved research are acknowledged, risk limits and portfolio impact have been reviewed, monitoring alerts have been checked, exit and invalidation criteria exist, the no-trade alternative was considered, and no active emergency condition exists.

### Governance is not a bypass

A completed checklist is not itself approval to trade. It improves review quality, but backend live gates still control execution.

## 16. Data Integrity, Backup, Restore, Import, and Migration

The data integrity layer exists because local-first software is only as durable as its runtime state. Use it to inventory local data, validate JSON/JSONL records, scan for secret-like content, create redacted backups, validate backup bundles, preview restore, import/export controlled bundles, and run migration dry-runs.

### Backup discipline

- Create a backup before risky configuration, migration, restore, import, or live-readiness changes.
- Use redacted backup defaults unless there is a very specific reason not to.
- Validate a backup before depending on it.
- Keep backups outside the release package.
- Do not include real .env files, private keys, auth headers, screenshots with secrets, logs with secrets, virtual environments, or build artifacts.

### Restore discipline

- Select the backup bundle.
- Validate the manifest.
- Preview compatibility, included subsystems, redaction policy, and restore impact.
- Create a pre-restore backup when practical.
- Apply restore only with explicit confirmation.
- Validate restored data and read the restore report.
- Confirm live mode, read-only state, and kill switch after restore.

## 17. Safe Demo Mode, Visual QA, and Release Validation

v3.3 improves the demo and visual QA workflow. Demo data is clearly fake, secret-free, local, and excluded from release ZIPs. Screenshot helpers and validation scripts help prepare a public demo or GitHub release without including runtime artifacts.

### Demo commands

```
PYTHONPATH=. python scripts/create_v3_demo_data.py
PYTHONPATH=. python scripts/clear_v3_demo_data.py
PYTHONPATH=. python scripts/capture_v3_screenshots.py --dry-run
PYTHONPATH=. python scripts/validate_v3_ux_release.py --quick
```

### Release validation commands

```
python -m compileall -q app tests scripts
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 PYTHONPATH=. python -m pytest -q
PYTHONPATH=. python scripts/check_versions.py
PYTHONPATH=. python scripts/smoke_startup.py
PYTHONPATH=. python scripts/validate_v3_ux_release.py
PYTHONPATH=. python scripts/validate_v3_release.py
python scripts/check_release_package.py .
```



**Screenshot handling**

Screenshots are runtime artifacts. Do not include screenshots in a release ZIP unless each screenshot is intentionally reviewed and confirmed secret-free.

## 18. Reports, Exports, and Documentation Outputs

The application can export many local artifacts as JSON, Markdown, or CSV. Exports are intended for review, audit, backup, and handoff. They must not contain secrets and should be treated as operational records.

### Common export categories

| Category       | Typical exports                                                                                |
|----------------|------------------------------------------------------------------------------------------------|
| Audit          | JSON, CSV, Markdown audit ledger views.                                                        |
| Strategy       | Strategy workspace JSON/Markdown and collection CSVs.                                          |
| Research       | Research export JSON/Markdown and source/queue/evidence CSVs.                                  |
| Monitoring     | Monitoring report JSON/Markdown and rules/alerts/history CSVs.                                 |
| Portfolio      | Portfolio report JSON/Markdown and exposure/warnings/scenarios CSVs.                           |
| Governance     | Governance report JSON/Markdown and journal/checklist/review/mistake CSVs.                     |
| Data integrity | Health/recovery reports and check CSVs.                                                        |
| v3             | Graph, workflow, packets, briefs, operator reviews, analytics snapshots, and learning reports. |

### Export rules

- Use exports for review and traceability, not as orders.
- Do not publish exports that contain private notes, account context, or operational details without review.
- Run a secret scan before sharing external artifacts.
- Prefer sanitized exports for GitHub issues or public demos.
- Keep runtime exports out of release ZIPs unless intentionally included and verified safe.

## 19. Recommended Operator Playbooks

### Daily startup playbook

- Start the app locally and log in.
- Open `/v3` and read the System Safety zone first.
- Confirm live armed, read-only, kill switch, readiness, and data health states.
- Review the Operator Attention Queue.
- Open active alerts, stale evidence, portfolio warnings, and incomplete governance items.
- Create a backup if data health, restore, import, migration, or large review work is planned.
- Run or update the daily operator review packet at the end of the session.

### Research-to-thesis playbook

- Add or review sources in Research.
- Create source notes and evidence candidates.
- Assess credibility, relevance, and freshness.
- Convert evidence candidates only after manual review.
- Create or update the thesis with support, counter-evidence, and unresolved questions.
- Add watchlist and monitoring reminders if action is not immediate.
- Run Thesis Health before considering ticket work.

## Pre-trade review playbook

- Confirm market and outcome context.
- Run Pre-Trade Intelligence Packet.
- Review thesis status, evidence coverage, counter-evidence, stale data, unresolved research, alerts, exposure, concentration, governance checklist, data health, and readiness.
- Review planned trade impact.
- Complete governance checklist.
- Use Live v2 ticket preview.
- Proceed only if backend gates intentionally pass and the operator still chooses to act.

## Post-trade or post-decision review playbook

- Record the decision journal entry or link the existing one.
- Update the thesis if the rationale, evidence, or invalidation criteria changed.
- Record post-trade review when outcome or fill/cancel data is available.
- Record lessons learned, near-misses, mistake patterns, and strength patterns.
- Update monitoring rules and portfolio assumptions.
- Add follow-up dates and review them in analytics.

## Weekly learning playbook

- Generate weekly operator review packet.
- Generate learning report.
- Review recurring mistake patterns and strength patterns.
- Review alert usefulness and noisy rules.
- Review stale evidence and unresolved research trends.
- Review governance discipline and incomplete follow-ups.
- Create process improvement items and link them to rules or checklists.

# 20. Troubleshooting and Recovery

## App will not start

- Confirm the virtual environment is active.
- Run ``pip install -r requirements.txt`` again.
- Check ``/setup/status`` after the app starts, or use startup smoke scripts if it does not.
- Confirm HOST and PORT are available.
- Inspect logs for dependency import failures.
- Use a clean virtual environment if dependencies are inconsistent.

## Cannot log in

- Confirm initial setup has been completed at ``/setup``.
- Confirm the account is active.
- Use an admin account to reset users if possible.
- Do not hand-edit user password hashes unless you understand the auth format.
- Restore from a backup if the local user store is corrupt.

## Data looks missing

- Confirm you are using the correct project directory and runtime data directory.
- Run data inventory and health checks.
- Check whether demo data was cleared.
- Check whether the release ZIP was used without runtime data, which is expected.
- Restore from a validated backup if records were removed unintentionally.

## Live readiness fails

- Treat failures as protective until proven otherwise.
- Read the specific failed readiness item.
- Confirm read-only state, kill switch state, live mode, network permission, credentials, optional SDK dependency, and risk settings.
- Do not bypass a failed readiness item through the UI.
- Use read-only verification where supported before any live-connected operation.

# 21. Route and API Reference

This section lists the most important routes for operators. API routes are included for debugging, automation around safe read-only workflows, and integration testing. Do not call live mutation endpoints without understanding the gates and compliance obligations.

## v3 UI routes

| Route                                               | Purpose                 |
|-----------------------------------------------------|-------------------------|
| <code>/v3</code> or <code>/v3/command-center</code> | Unified command center. |
| <code>/v3/search</code>                             | Global local search.    |

| Route                         | Purpose                                    |
|-------------------------------|--------------------------------------------|
| /v3/graph                     | Decision graph explorer.                   |
| /v3/workflows                 | Read-only workflow orchestrator.           |
| /v3/briefs                    | Packets, reports, and briefs.              |
| /v3/analytics                 | Analytics dashboard.                       |
| /v3/analytics/learning-report | Learning report view.                      |
| /v3/settings                  | v3 settings and safe AI/provider boundary. |
| /v3/docs                      | Documentation index.                       |

## v3 API route families

| Family                | Examples                                                                                                                                                          |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Command center and UX | /api/v3, /api/v3/command-center, /api/v3/ux/status, /api/v3/ux/navigation                                                                                         |
| Search                | /api/v3/search, /api/v3/search/index, /api/v3/search/rebuild, /api/v3/search/filters                                                                              |
| Graph                 | /api/v3/graph, /api/v3/graph/rebuild, /api/v3/graph/export.json, /api/v3/graph/export.md                                                                          |
| Workflows and reports | /api/v3/workflows, /api/v3/workflows/run, /api/v3/pre-trade-packet, /api/v3/market-brief, /api/v3/thesis-health, /api/v3/portfolio-brief, /api/v3/operator-review |
| Demo and validation   | /api/v3/demo/create, /api/v3/demo/clear, /api/v3/demo/status, /api/v3/validation/status                                                                           |
| Analytics             | /api/v3/analytics, /api/v3/analytics/summary, /api/v3/analytics/snapshot, /api/v3/analytics/learning-report, /api/v3/analytics/export.*                           |

## v2-live module route families

| Family                          | Examples                                                                                      |
|---------------------------------|-----------------------------------------------------------------------------------------------|
| Status/readiness                | /api/v2/live/status, /api/v2/live/readiness, /api/v2/live/demo-readiness                      |
| Markets/order books             | /api/v2/live/markets, /api/v2/live/orderbook/{token_id}                                       |
| Ticket/submit/cancel            | /api/v2/live/ticket/preview, /api/v2/live/order/submit, /api/v2/live/order/cancel             |
| Audit/verify/settings/emergency | /api/v2/live/audit, /api/v2/live/verify, /api/v2/live/settings/schema, /api/v2/live/emergency |
| Strategy                        | /api/v2/live/strategy/*                                                                       |
| Research                        | /api/v2/live/research/*                                                                       |
| Monitoring                      | /api/v2/live/monitoring/*                                                                     |
| Portfolio                       | /api/v2/live/portfolio/*                                                                      |
| Governance                      | /api/v2/live/governance/*                                                                     |
| Data integrity                  | /api/v2/live/data/*                                                                           |

## 22. Appendix: Environment Variable Reference

This appendix summarizes important environment groups. It is not a substitute for ``.env.example`` or the schema-backed configuration console. Never copy real secrets into shared files, manuals, screenshots, or tickets.

### Server and security

| Variable                                             | Operator meaning                                                                 |
|------------------------------------------------------|----------------------------------------------------------------------------------|
| HOST                                                 | Bind address. Use 127.0.0.1 for local-only, 0.0.0.0 for LAN access.              |
| PORT                                                 | HTTP port, usually 8000.                                                         |
| APP_RELOAD                                           | Development reload behavior. Avoid in production-like demos unless needed.       |
| ALLOWED_HOSTS                                        | Host allow-list for LAN/security posture.                                        |
| SECURITY_HEADERS_ENABLED                             | Enables security headers. Keep enabled unless debugging.                         |
| SESSION_COOKIE_SECURE and<br>SESSION_COOKIE_SAMESITE | Session cookie security posture. Configure carefully for HTTPS and LAN contexts. |

## Live and read-only controls

| Variable                                                   | Operator meaning                                                                   |
|------------------------------------------------------------|------------------------------------------------------------------------------------|
| APP_MODE                                                   | Overall app mode; read_only is safest.                                             |
| READ_ONLY                                                  | Global read-only posture. True blocks live mutation paths.                         |
| LIVE_TRADING_ENABLED                                       | High-level live feature toggle. Not sufficient by itself for execution.            |
| POLYMARKET_V2_TRADING_MODE                                 | v2 control-plane mode such as research_only or live_trading_armed.                 |
| POLYMARKET_V2_REQUIRE_APPROVAL                             | Requires explicit human approval in v2 workflow.                                   |
| POLYMARKET_V2_CONFIRMATION_PHRASE                          | Typed phrase required for gated live actions.                                      |
| POLYMARKET_LIVE_KILL_SWITCH                                | Emergency kill switch. True blocks live mutation.                                  |
| POLYMARKET_LIVE_ALLOW_REAL_NETWORK                         | Explicit permission for real network operations. Keep false for normal validation. |
| POLYMARKET_LIVE_ENABLE_SUBMIT /<br>CANCEL                  | Enables submit/cancel capability only when all other gates also pass.              |
| POLYMARKET_LIVE_MANUAL_SUBMIT_ENAB<br>LED / CANCEL_ENABLED | Manual live execution toggles. Default false.                                      |

## Risk and paper settings

| Variable                                                                           | Operator meaning                                                         |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| PAPER_MAX_STAKE_PER_TRADE                                                          | Paper trade per-trade cap.                                               |
| PAPER_MAX_MARKET_EXPOSURE                                                          | Paper market exposure cap.                                               |
| PAPER_MAX_TOTAL_EXPOSURE                                                           | Paper total exposure cap.                                                |
| PAPER_MAX_OPEN_POSITIONS                                                           | Paper open-position cap.                                                 |
| PAPER_MIN_LIQUIDITY and<br>PAPER_MIN_VOLUME_24HR                                   | Paper workflow liquidity/volume thresholds.                              |
| LIVE_MAX_ORDER_NOTIONAL /<br>LIVE_MAX_MARKET_NOTIONAL /<br>LIVE_MAX_DAILY_NOTIONAL | Live-readiness risk caps. Zero defaults keep live execution fail-closed. |

## Data ingestion and training

| Variable                        | Operator meaning                                              |
|---------------------------------|---------------------------------------------------------------|
| POLYMARKET_DATA_ALLOW_NETWORK   | Allows data network access when intentionally enabled.        |
| POLYMARKET_DATA_ALLOW_INTERNET  | Allows broader internet ingestion when intentionally enabled. |
| POLYMARKET_DATA_ALLOWED_DOMAINS | Domain allow-list for scoped ingestion.                       |

| Variable                                             | Operator meaning                                                     |
|------------------------------------------------------|----------------------------------------------------------------------|
| POLYMARKET_DATA_MAX_REQUESTS_PER_RUN                 | Caps internet/data requests per run.                                 |
| POLYMARKET_DATA_STORE_RAW_RESPONSES                  | Controls whether raw responses are stored. Keep false unless needed. |
| POLYMARKET_TRAINING_HOST_JOBS_ENABLED                | Host training jobs disabled by default.                              |
| POLYMARKET_TRAINING_DEFAULT_MAX_ROWS / HARD_MAX_ROWS | Training row caps.                                                   |

## Credential placeholders

Variables such as `POLY\_PRIVATE\_KEY`, `POLYMARKET\_PRIVATE\_KEY`, `POLYMARKET\_CLOB\_API\_KEY`, `CLOB\_API\_KEY`, `POLYMARKET\_CLOB\_SECRET`, `CLOB\_SECRET`, `POLYMARKET\_CLOB\_PASSPHRASE`, and related wallet/API fields must be populated only in private local runtime configuration when the operator intentionally configures live functionality. Never include real values in release assets.

# 23. Appendix: Maintenance, Upgrades, and Rollback

## Before upgrading

- Create a redacted runtime backup.
- Export a sanitized configuration snapshot.
- Record current app version and package SHA.
- Run data health checks.
- Clear demo data if it should not be carried forward.
- Confirm no live operation is in progress.

## After upgrading

- Start the app and log in.
- Open `/v3` and confirm the version label.
- Run `/setup/status` or the startup smoke script.
- Run data health checks.
- Rebuild search/graph indexes if needed.
- Run a workflow packet and verify output readability.
- Confirm v2 pages still render.
- Confirm live/read-only/kill switch states are exactly what you expect.

## Rollback

Rollback is package and data dependent. Keep the prior ZIP and runtime backup until the new version is validated. If a new version writes migration changes, use the data layer's preview and report outputs to understand reversibility before applying migrations. Never roll back blindly over newer runtime data without a backup.

# 24. Closing Operator Notes

Polymarket Gamma Starter v3.3.0-real is a powerful local-first operator workstation. Its value comes from structure: thesis discipline, evidence hygiene, monitoring awareness, portfolio context, governance follow-through, data backups, analytics, and cautious execution boundaries. Use the software to slow down, organize evidence, notice missing prerequisites, and create a durable decision trail.

**Final reminder**

The safest default is read-only. Do not move toward live mutation unless you understand the configuration, the platform rules, the risks, the backend gates, and the full operational consequences.